

PRACTICAL NO:2

CYBER INFORMATION AND SECURITY

AIM: Implement the RSA algorithm for public-key encryption and decryption, and explore its properties and security considerations.

CODE: COMMAND LINE RSA ALGORITHM

```
import math

# Function to find GCD
def gcd(a, b):
    while b != 0:
        a, b = b, a % b
    return a

# Function to find Modular Inverse
def mod_inverse(e, phi):
    for d in range(1, phi):
        if (e * d) % phi == 1:
            return d
    return None

# Function to check Prime Number
def prime(n):
    if n <= 1:
        return False
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            return False
    return True

# ----- MAIN PROGRAM -----

# Input Prime Numbers
p = int(input("Enter first prime number (p): "))
q = int(input("Enter second prime number (q): "))

# Check Prime Numbers
if not (prime(p) and prime(q)):
    print("Error: Enter Prime Numbers Only!")
```

PRACTICAL NO:2

CYBER INFORMATION AND SECURITY

```
exit()

# RSA Calculations
n = p * q
phi = (p - 1) * (q - 1)

# Choose Public Key e
e = 2
while e < phi:
    if gcd(e, phi) == 1:
        break
    e += 1

# Calculate Private Key d
d = mod_inverse(e, phi)

# Display Keys
print("\n===== RSA KEYS =====")
print("Public Key (e, n):", (e, n))
print("Private Key (d, n):", (d, n))

# Input Message
message = int(input("\nEnter message (number less than n): "))

# Validate Message
if message >= n:
    print("Error: Message must be less than", n)
    exit()

# Encryption
cipher = pow(message, e, n)
print("\nEncrypted Message:", cipher)

# Decryption
decrypted = pow(cipher, d, n)
print("Decrypted Message:", decrypted)

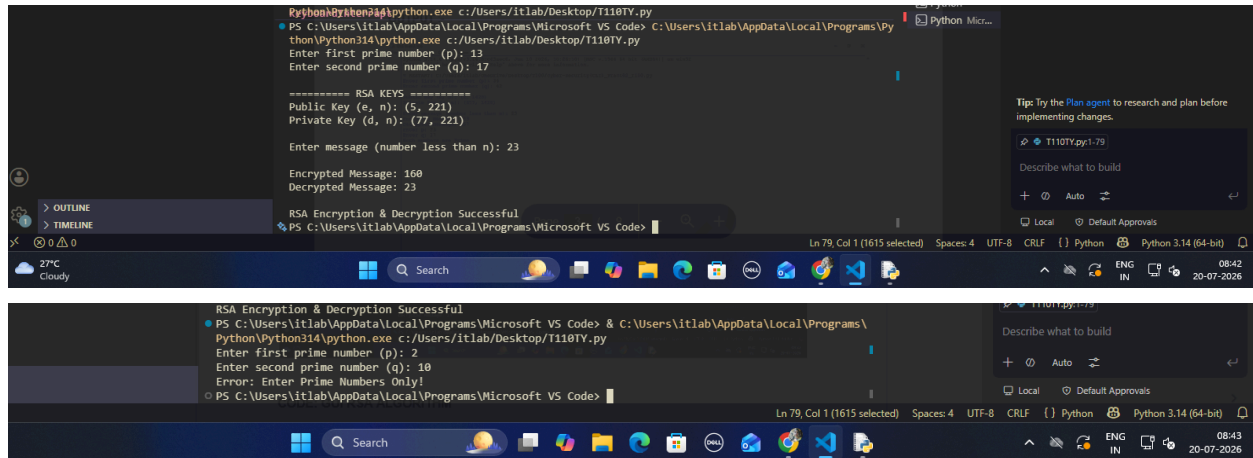
# Verification
if decrypted == message:
    print("\nRSA Encryption & Decryption Successful")
```

PRACTICAL NO:2

CYBER INFORMATION AND SECURITY

```
else:  
    print("\nRSA Failed")
```

OUTPUT:



The first screenshot shows the terminal output of the RSA algorithm. It prompts the user to enter the first prime number (p) as 13 and the second prime number (q) as 17. It then displays the calculated RSA keys: Public Key (e, n): (5, 221) and Private Key (d, n): (77, 221). The user enters a message of 23, which is encrypted to 160 and then decrypted back to 23. The second screenshot shows an error message: "Error: Enter Prime Numbers Only!" because the user entered p=2 and q=10, which are not prime numbers.

```
Python314\python.exe c:/Users/itlab/Desktop/T110TY.py  
PS C:\Users\itlab\AppData\Local\Programs\Microsoft VS Code> C:\Users\itlab\AppData\Local\Programs\Python\Python314\python.exe c:/Users/itlab/Desktop/T110TY.py  
Enter first prime number (p): 13  
Enter second prime number (q): 17  
----- RSA KEYS -----  
Public Key (e, n): (5, 221)  
Private Key (d, n): (77, 221)  
Enter message (number less than n): 23  
Encrypted Message: 160  
Decrypted Message: 23  
RSA Encryption & Decryption Successful  
PS C:\Users\itlab\AppData\Local\Programs\Microsoft VS Code>  
  
RSA Encryption & Decryption Successful  
PS C:\Users\itlab\AppData\Local\Programs\Microsoft VS Code> C:\Users\itlab\AppData\Local\Programs\Python\Python314\python.exe c:/Users/itlab/Desktop/T110TY.py  
Enter first prime number (p): 2  
Enter second prime number (q): 10  
Error: Enter Prime Numbers Only!  
PS C:\Users\itlab\AppData\Local\Programs\Microsoft VS Code>
```

CODE: GUI RSA ALGORITHM

```
print("T110 SANJANA SAHU TYCS")  
from tkinter import *  
from tkinter import messagebox  
  
# ----- FUNCTIONS ----- #  
  
def gcd(a, b):  
    while b:  
        a, b = b, a % b  
    return a  
  
def mod_inverse(e, phi):  
    for d in range(1, phi):  
        if (e * d) % phi == 1:  
            return d  
    return None  
  
def is_prime(num):
```

T110 SANJANA SAHU TYCS

PRACTICAL NO:2

CYBER INFORMATION AND SECURITY

```
if num <= 1:
    return False

for i in range(2, int(num ** 0.5) + 1):
    if num % i == 0:
        return False

return True

def encrypt():
    try:

        p = int(entry_p.get())
        q = int(entry_q.get())
        msg = int(entry_msg.get())

        if not is_prime(p) or not is_prime(q):
            messagebox.showerror("Error", "Please enter valid prime
numbers.")
            return

        n = p * q

        if msg >= n:
            messagebox.showerror("Error", "Message must be smaller than
n.")
            return

        phi = (p - 1) * (q - 1)

        e = 2
        while e < phi:
            if gcd(e, phi) == 1:
                break
            e += 1

        d = mod_inverse(e, phi)

        if d is None:
```

PRACTICAL NO:2

CYBER INFORMATION AND SECURITY

```
        messagebox.showerror("Error", "Couldn't generate private
key.")

        return

    cipher = pow(msg, e, n)

    lbl_public.config(text=f"({e}, {n})")
    lbl_private.config(text=f"({d}, {n})")
    lbl_cipher.config(text=str(cipher))
    lbl_plain.config(text="")

    btn_decrypt.config(command=lambda: decrypt(cipher, d, n))

except ValueError:
    messagebox.showerror("Invalid Input", "Please enter only
numbers.")

def decrypt(cipher, d, n):
    plain = pow(cipher, d, n)
    lbl_plain.config(text=str(plain))

def clear():
    entry_p.delete(0, END)
    entry_q.delete(0, END)
    entry_msg.delete(0, END)

    lbl_public.config(text="")
    lbl_private.config(text="")
    lbl_cipher.config(text="")
    lbl_plain.config(text="")

# ----- WINDOW ----- #

root = Tk()

root.title("RSA Encryption & Decryption")
root.geometry("900x650")
```

PRACTICAL NO:2

CYBER INFORMATION AND SECURITY

```
root.configure(bg="#EAF4FC")

# ----- MENU ----- #

menu = Menu(root)
root.config(menu=menu)

file_menu = Menu(menu, tearoff=0)
menu.add_cascade(label="File", menu=file_menu)

file_menu.add_command(label="Clear", command=clear)
file_menu.add_separator()
file_menu.add_command(label="Exit", command=root.destroy)

# ----- TITLE ----- #

title = Label(
    root,
    text="RSA PUBLIC KEY ENCRYPTION & DECRYPTION",
    bg="#0D47A1",
    fg="white",
    font=("Arial", 22, "bold"),
    pady=15
)

title.pack(fill=X)

# ----- MAIN FRAME ----- #

main = Frame(root, bg="#EAF4FC")
main.pack(fill=BOTH, expand=True, padx=30, pady=30)

card = Frame(main, bg="white", bd=3, relief=RIDGE)
card.place(relx=0.5, rely=0.5, anchor="center")

# ----- INPUT ----- #

Label(card, text="Prime Number (p)", bg="white",
      font=("Arial", 12, "bold")).grid(row=0, column=0, padx=20, pady=15)
```

PRACTICAL NO:2

CYBER INFORMATION AND SECURITY

```
entry_p = Entry(card, font=("Arial", 12), width=30)
entry_p.grid(row=0, column=1)

Label(card, text="Prime Number (q)", bg="white",
       font=("Arial", 12, "bold")).grid(row=1, column=0, padx=20, pady=15)

entry_q = Entry(card, font=("Arial", 12), width=30)
entry_q.grid(row=1, column=1)

Label(card, text="Message", bg="white",
       font=("Arial", 12, "bold")).grid(row=2, column=0, padx=20, pady=15)

entry_msg = Entry(card, font=("Arial", 12), width=30)
entry_msg.grid(row=2, column=1)

# ----- BUTTONS ----- #

btn_frame = Frame(card, bg="white")
btn_frame.grid(row=3, column=0, columnspan=2, pady=20)

Button(
    btn_frame,
    text="Encrypt",
    bg="#1976D2",
    fg="white",
    width=12,
    font=("Arial", 12, "bold"),
    command=encrypt
).grid(row=0, column=0, padx=10)

btn_decrypt = Button(
    btn_frame,
    text="Decrypt",
    bg="#2E7D32",
    fg="white",
    width=12,
    font=("Arial", 12, "bold")
)

btn_decrypt.grid(row=0, column=1, padx=10)
```

PRACTICAL NO:2

CYBER INFORMATION AND SECURITY

```
Button(
    btn_frame,
    text="Clear",
    bg="#F57C00",
    fg="white",
    width=12,
    font=("Arial", 12, "bold"),
    command=clear
).grid(row=0, column=2, padx=10)

# ----- OUTPUT ----- #

Label(card, text="Public Key", bg="white",
      fg="#1565C0", font=("Arial", 12, "bold")).grid(row=4, column=0,
sticky="w", padx=20)

lbl_public = Label(card, bg="white", font=("Arial", 12))
lbl_public.grid(row=4, column=1, sticky="w")

Label(card, text="Private Key", bg="white",
      fg="#1565C0", font=("Arial", 12, "bold")).grid(row=5, column=0,
sticky="w", padx=20)

lbl_private = Label(card, bg="white", font=("Arial", 12))
lbl_private.grid(row=5, column=1, sticky="w")

Label(card, text="Encrypted Message", bg="white",
      fg="#1565C0", font=("Arial", 12, "bold")).grid(row=6, column=0,
sticky="w", padx=20)

lbl_cipher = Label(card, bg="white", font=("Arial", 12))
lbl_cipher.grid(row=6, column=1, sticky="w")

Label(card, text="Decrypted Message", bg="white",
      fg="#1565C0", font=("Arial", 12, "bold")).grid(row=7, column=0,
sticky="w", padx=20)

lbl_plain = Label(card, bg="white", fg="red",
                  font=("Arial", 12, "bold"))
```


PRACTICAL NO:2

CYBER INFORMATION AND SECURITY

```
lbl_plain.grid(row=7, column=1, sticky="w")

# ----- FOOTER ----- #

footer = Label(
    root,
    text="Developed using Python Tkinter | RSA Cryptography Practical",
    bg="#0D47A1",
    fg="white",
    font=("Arial", 10)
)

footer.pack(fill=X, side=BOTTOM)

root.mainloop()
```

OUTPUT:

